Do Language Models Return Every Answer?

A Benchmark for Completeness in Enumeration and Relational Operations

Daryl Alfaro

October 12, 2026

Abstract

When a user asks a language model for *all* items that satisfy a condition, a correct-looking answer can still be incomplete. We introduce a 500-question benchmark that measures this directly. Questions cover enumeration and six SQL-like operations (selection, projection, JOIN, aggregation, nested queries, set operations) over synthetic records whose exact answers are obtained by executing SQL, so every omission can be counted. We evaluate five models from three families on 365 questions whose records are given in the prompt, plus memory-only and retrieval variants. Mean recall is uniformly high (0.966–0.996), yet the share of answers that are exactly the correct set ranges from 98% to 81%, and almost all errors are omissions: models rarely add wrong items, they leave correct ones out. For three of five models the probability of a complete answer falls significantly as the answer set grows (for Claude Haiku 5.5, from 100% with at most 10 correct items to 53% with 50 or more). Instructions to be exhaustive or to check every record produce no significant improvement, and the models that remain complete spend several times more reasoning tokens on larger answer sets. We conclude that recall overstates reliability for "list all" requests, that exact-set accuracy as a function of answer size is the informative measure, and that completeness failures are measurable and model-dependent but, for the strongest models, near ceiling on this version of the benchmark.

1 Introduction

Language models are increasingly used to answer questions over data that is placed in their context: tables pasted into a prompt, documents returned by a retriever, or records produced by a tool. Many such questions ask for a *set*: every customer in a region, every order above a threshold, every pair of records that match. For these requests an answer can be fluent, plausible and entirely accurate in what it lists while still being wrong, because it is incomplete. Standard question-answering metrics hide this failure: an answer that lists 47 of 50 correct items has a recall of 0.94 and a precision of 1.0, and nothing in it signals that three items are missing.

This report studies **completeness**: whether a model returns the full answer set when the information needed to do so is available. We ask five research questions:

RQ1 How often are answers complete, and what kinds of errors do models make?

RQ2 Does completeness degrade as the number of correct items grows, independently of how much data the model reads?

RQ3 Do failures generalise across relational operations?

RQ4 Are failures systematic (the same question fails repeatedly) or run-to-run variation?

RQ5 Can prompting fix it, and how does reasoning effort relate to completeness?

Contributions. (1) A benchmark of 500 questions in which the number of correct items is controlled separately from table size, and every gold answer is produced by executing SQL rather than by annotation. (2) A set-level error taxonomy (missing, near-miss, wrong, made-up, duplicate) that distinguishes omission from hallucination without a model-based judge. (3) An evaluation of five models with confidence intervals, paired

comparisons and repeated samples, following current recommendations for reporting evaluation results [12, 14]. (4) Released code, data, raw responses and run manifests.

Key findings.

1. **Recall hides incompleteness.** Mean recall is 0.966–0.996 for every model, but only 81–98% of answers are exactly the correct set; 2–15% of answers omit at least one correct item. (RQ1)
2. **The dominant error is omission.** Four of the five models together returned 25 wrong items and 0 invented ones across 365 questions each; the fifth (MiMo-V2.6 Flash) invented 1,032 items, 991 of them on a single question. (RQ1)
3. **Completeness falls with answer size.** For three models the odds of a complete answer drop to 0.3–0.4× with each doubling of the number of correct items. (RQ2)
4. **The effect is general but uneven across operations,** largest for enumeration and near zero for set operations. (RQ3)
5. **Some failures are systematic, many are not.** Two independent samples agree on 87–100% of verdicts; some failures recur on every re-run, others never do. (RQ4)
6. **Prompting does not fix it;** models that stay complete spend far more reasoning on large answer sets. (RQ5)

2 Related work

The closest work is HoloBench [10], which verbalises database rows into long contexts and evaluates five query types while varying context length and information density; it scores answers with a model-based judge and partial credit, and does not vary the number of correct items independently. LOFT [8] tests whether long-context models can replace retrieval and SQL and finds that compositional SQL-like reasoning remains difficult. Oolong [4] targets long-context aggregation (counting, distributions) with scalar answers, and LongProc [1] includes HTML-to-TSV extraction scored by row-level F1, which penalises dropped rows but involves no selection condition. Multi-answer question answering benchmarks (QAMPARI [3], RoMQA [18], QUEST [11]) evaluate list answers over real corpora, where retrieval, missing knowledge and reading failures are confounded. Galois [15] executes SQL against a model's parametric knowledge and reports low recall. Long-context diagnostics study related effects: position sensitivity [9], retrieving all values for a key [6], reasoning over facts hidden in long text [7], degradation when lists exceed roughly 100 items [16], locating items by position [17], and model-specific counting failures [2]. Systems work instead *engineers* completeness through query planning [5, 13]. Relative to this literature, our benchmark isolates answer-set size as a controlled variable, uses exact executable ground truth with a set-level error taxonomy, and covers seven operation types in one framework.

3 Benchmark design

3.1 Synthetic data with executable ground truth

All data is generated from a fixed seed: 575 tables containing 110,146 records about fictional employees, products, students, books, customers, orders, projects, ID badges and club rosters. Names, cities, brands and titles are invented, so the model cannot answer from memory and must read the records. Each question is generated together with the SQL query that defines it; its gold answer is the result of running that query in SQLite, and an automated test re-executes every query against the stored answer. Each filtering condition is produced in SQL and in English from a single specification, so the two cannot disagree.

3.2 Controlled factors

For in-context enumeration ("List the names of all employees whose department is Legal"), exactly g rows of an n-row table receive the target value, with $g \in \{5, 10, 20, 50, 100\}$, $n \in \{50, 200, 500, 1000\}$ and $g \leq n/2$; matching rows are placed at random positions, which are recorded. For the other operations, the generator

Slice	Qs	Variants	Answer size	Context (k tok.)
Enumeration	100	one category filter	5–100	7.6
Enumeration (memory only)	60	U.S. states, elements, presidents	2–50	0.0
Enumeration (retrieval)	40	BM25 top-k, $k \in \{10, 20, 50\}$	5–40	1.1
Selection	60	1–3 conditions	1–92	4.1
Projection	40	DISTINCT; 2–3 attributes	7–39	2.9
JOIN	60	1:1, 1:N, M:N	3–55	3.4
Aggregation	50	COUNT, SUM, MIN/MAX/AVG, GROUP BY	3–367	2.8
Nested query	40	IN, NOT IN, correlated, scalar	3–32	2.7
Set operation	50	UNION, INTERSECT, EXCEPT, 3-way	4–97	0.7

Table 1: Benchmark composition (500 questions). Answer size is the number of correct items (for aggregation, the number of records aggregated). Context is the median size of the data in the prompt.

samples queries from templates, executes them, and keeps the first whose answer size falls in a target range. Records are rendered as Markdown, CSV, JSON Lines or English sentences, rotating across questions.

3.3 Composition

Table 1 lists the slices. The 400 *in-context* questions place all records in the prompt, so any omission is attributable to the model; they are the core of the evaluation. Two further slices test other access paths: 60 *memory-only* questions filter closed lists (U.S. states, chemical elements, presidents' surnames) by properties computed in code, and 40 *retrieval* questions supply the top-k records returned by BM25.

4 Evaluation protocol

4.1 Models and access

We evaluate DeepSeek V4 Pro and V4 Flash (via the OpenCode Go API), Claude Haiku 5.5 (via the Claude Code CLI, version 2.1.296, with no tools and a fixed system prompt) and MiMo-V2.6 Pro and Flash (via OpenRouter). API calls use temperature 0 and a 32,768-token output limit; the CLI does not expose temperature. All models use their default reasoning behaviour. Each prompt contains the records, the question and a required JSON answer format.

4.2 Metrics

For a question with gold set G and predicted set P (after normalising case, accents, whitespace and number formatting, and removing duplicates):

$$\text{recall} = \frac{|P \cap G|}{|G|}, \qquad \text{precision} = \frac{|P \cap G|}{|P|}, \qquad \text{exact-set accuracy} = \mathbb{1}[P = G].$$

Exact-set accuracy is our primary measure: it is the probability that a user who asks for every matching item receives exactly that. Every error is classified. An item of $G \setminus P$ is *missing*; an item of $P \setminus G$ is a *near-miss* if it is a garbled copy of a missing item (string similarity ≥ 0.85), *wrong* if it exists in the data but does not satisfy the question, and *made-up* if it does not exist in the data; repeated items are *duplicates*. An answer *omits* if it has a missing or near-miss item, and *commits* if it has a wrong, made-up or duplicate item. Scalar aggregates are scored by exact match and the sign of the error.

4.3 Experiments and statistics

E1 runs all 500 questions once per model with a direct prompt. **E2** runs a stratified 100-question subset (every fifth question) with two alternative prompts: an explicit instruction to be exhaustive, and an instruction to check every record before answering. **E3** runs the same subset with the direct prompt a second time.

Model	Exact-set acc. (%)	Δ vs. best (pp)	Mean recall	Mean prec.	Omit. (%)	Comm. (%)
DeepSeek V4 Pro	98 [96, 99]	−0.3 [-2.5, +1.6]	0.996 [0.991, 1.000]	0.997	2	0
DeepSeek V4 Flash	98 [96, 99]	(reference)	0.996 [0.989, 1.000]	0.996	2	1
Claude Haiku 5.5	91 [88, 93]	−7.1 [-10.1, -4.1]	0.995 [0.993, 0.997]	0.999	9	1
MiMo-V2.6 Pro	90 [86, 93]	−8.2 [-11.8, -4.9]	0.987 [0.978, 0.994]	0.992	7	4
MiMo-V2.6 Flash	81 [77, 85]	−16.7 [-20.8, -12.3]	0.966 [0.952, 0.979]	0.970	15	10

Table 2: RQ1. In-context set-valued questions (E1, $n = 365$ per model). Brackets are 95% intervals. Δ is the paired difference in exact-set accuracy from the best model on the same questions. *Omit.*: share of answers missing at least one correct item. *Comm.*: share adding at least one incorrect or duplicate item.

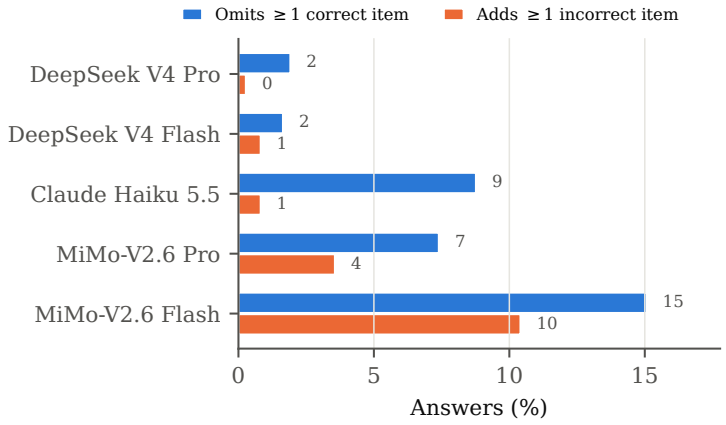


Figure 1: RQ1. Share of in-context answers that omit at least one correct item, and that add at least one incorrect item (E1).

Following Miller [12] we report 95% Wilson intervals for proportions and question-level bootstrap intervals (2,000 resamples) for means; differences between models or prompts are computed as *paired* question-level differences. Trends with answer size are estimated by a logistic regression of exact-set accuracy on $\log_2$ of the number of correct items, reported as an odds ratio per doubling with a bootstrap interval. Each answer-size cell contains 16–23 questions per model, so per-cell estimates carry wide intervals.

5 Results

5.1 RQ1: How often are answers complete?

Table 2 reports results on the 365 in-context set-valued questions. Recall and precision are close to 1 for every model, but exact-set accuracy separates them clearly. The two DeepSeek models are complete on 98% of questions and statistically indistinguishable from each other; Claude Haiku 5.5, MiMo-V2.6 Pro and MiMo-V2.6 Flash are significantly lower, with paired differences of −7 to −17 percentage points whose intervals all exclude zero.

The errors are overwhelmingly of one kind (Figure 1; counts in Appendix Table 8). For four of the five models, omissions are two to eleven times as frequent as commissions, and precision stays at or above 0.99: when these models answer, what they list is almost always right, and what goes wrong is what they leave out. MiMo-V2.6 Flash is the exception. On long lists it invents plausible items, typically common first-and-last-name combinations, adding 1,032 non-existent items in total and 991 on a single 100-item question.

Other access paths. In the memory-only setting both DeepSeek models answered every question exactly; the other three missed members of short closed lists (Appendix Table 9). Claude Haiku 5.5, for example, listed Kentucky but not New Jersey among U.S. states ending in “y”. In the retrieval setting every model

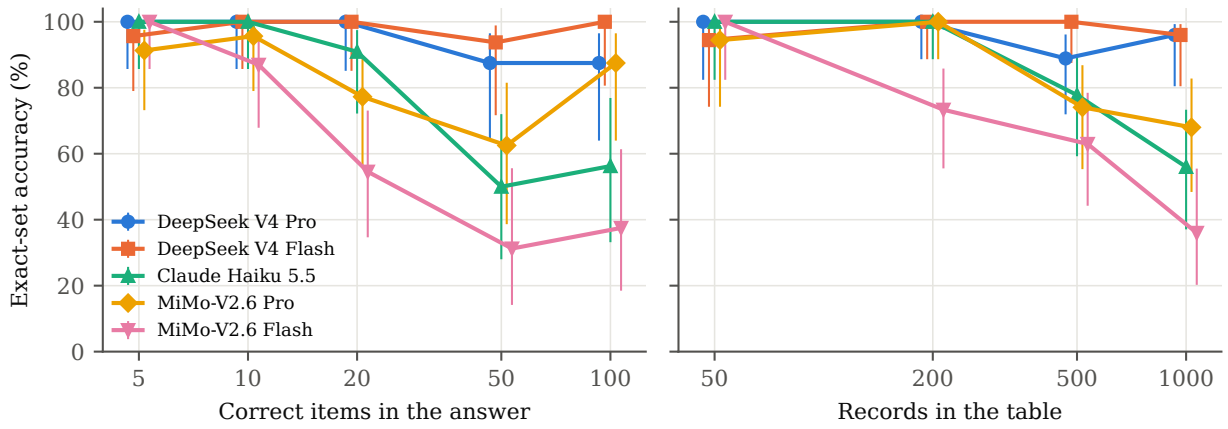


Figure 2: RQ2. In-context enumeration (E1): exact-set accuracy against the number of correct items (left) and the number of records in the table (right), with 95% Wilson intervals. Points are offset horizontally for legibility.

Model	≤10 items (%)	≥50 items (%)	Odds ratio per doubling [95% CI]
DeepSeek V4 Pro	100	88	0.31 [0.05, 0.54]
DeepSeek V4 Flash	98	97	not estimable (2 failures)
Claude Haiku 5.5	100	53	0.32 [0.17, 0.45]
MiMo-V2.6 Pro	93	75	0.74 [0.51, 1.05]
MiMo-V2.6 Flash	93	34	0.41 [0.26, 0.55]

Table 3: RQ2. Exact-set accuracy on in-context enumeration by answer size, and the odds ratio of a complete answer per doubling of the number of correct items (logistic regression; values below 1 mean degradation).

returned every correct record that BM25 retrieved, so recall there (0.869 for all models) equals the retriever's recall: with this configuration the retrieval slice measures the retriever, not the model.

5.2 RQ2: Does completeness degrade with answer size?

Figure 2 plots exact-set accuracy on in-context enumeration against the number of correct items and against table size. With at most 10 correct items every model is complete on at least 93% of questions; with 50 or more, Claude Haiku 5.5 falls to 53%, MiMo-V2.6 Flash to 34% and MiMo-V2.6 Pro to 75% (Table 3). The logistic trend is significant for DeepSeek V4 Pro, Claude Haiku 5.5 and MiMo-V2.6 Flash, whose odds of a complete answer fall to 0.31–0.41× per doubling of the answer set. For MiMo-V2.6 Pro the interval includes no effect, and DeepSeek V4 Flash fails too rarely for a trend to be estimated. Table size relates less strongly to completeness than answer size does, except for MiMo-V2.6 Flash.

5.3 RQ3: Do failures generalise across operations?

Table 4 breaks exact-set accuracy down by operation. Every operation except set operations yields incomplete answers from at least three models. Enumeration is the hardest slice because it contains the largest answer sets (up to 100 items), while set operations over club rosters are solved almost perfectly by every model even with up to 97 correct items. On the 35 scalar aggregates (COUNT, SUM, MIN, MAX, AVG; Appendix Table 10) both DeepSeek models are always exact, and 9 of the other models' 11 errors are undercounts, consistent with items being dropped before counting.

5.4 RQ4: Are failures systematic?

A second independent sample of the subset gives the same exact/inexact verdict on 87–100% of questions (Table 5); between 0 and 12 questions per model flip. To test individual failures, we re-ran 13 failed questions

Operation	Qs	DeepSeek V4 Pro	DeepSeek V4 Flash	Claude Haiku 5.5	MiMo-V2.6 Pro	MiMo-V2.6 Flash
Enumeration	100	96	98	83	84	66
Selection	60	97	98	95	93	88
Projection	40	100	95	90	90	82
JOIN	60	100	98	93	88	82
Aggregation	15	93	93	93	87	67
Nested query	40	98	100	90	92	90
Set operation	50	100	100	100	98	100

Table 4: RQ3. Exact-set accuracy (%) on in-context set-valued questions by operation (E1). *Qs*: questions per model. Aggregation here covers GROUP BY questions; scalar aggregates are in Appendix Table 10.

Model	Qs	Same verdict (%)	Questions that flipped
DeepSeek V4 Pro	87	98 [92, 99]	2
DeepSeek V4 Flash	93	100 [96, 100]	0
Claude Haiku 5.5	93	94 [87, 97]	6
MiMo-V2.6 Pro	93	91 [84, 96]	8
MiMo-V2.6 Flash	93	87 [79, 92]	12

Table 5: RQ4. Agreement between two independent samples of the direct prompt on the subset (E1 vs. E3; set-valued questions with a valid response in both), with 95% Wilson intervals.

three more times on the same model (Table 6). Some failures recur every time: Claude Haiku 5.5 omitted the same U.S. states from memory in all three re-runs, and DeepSeek V4 Pro and MiMo-V2.6 Flash both dropped items from the same 50-item list in every re-run. When MiMo-V2.6 Pro failed on that list, it omitted the same three employees each time. Other failures, including Claude Haiku 5.5's misses on selection, JOIN, projection and COUNT questions, did not recur. Completeness is therefore best measured as a *rate*, estimated over many questions and repeated samples, rather than as a fixed set of questions that always fail.

5.5 RQ5: Prompting and reasoning

Neither alternative prompt produced a significant change in exact-set accuracy for any model: every paired interval in Table 7 includes zero. Reasoning effort, by contrast, tracks answer size (Figure 3). Between the smallest and largest answer sets, the median reasoning length grows from 1,050 to 8,832 tokens for DeepSeek V4 Pro and from 173 to 6,442 for Claude Haiku 5.5. MiMo-V2.6 Flash often answers after a single sentence of reasoning: 61 of its 100 enumeration answers used fewer than 100 reasoning tokens, and these were complete 62% of the time, against 72% for the rest. That is a weak association and not evidence that reasoning effort causes completeness, since the model chooses its own reasoning length.

6 Discussion: what the results mean

Recall is the wrong headline metric for "list all" requests. A recall of 0.995 sounds close to perfect, yet the model with that recall (Claude Haiku 5.5) returns an incomplete answer to about one question in eleven, and to nearly half of questions with 50 or more correct items. A user cannot tell which answers are incomplete, because the items that are listed are almost always correct. Exact-set accuracy, reported as a function of answer size, is the measure that matches the user's experience.

The failure is omission, which is harder to detect than hallucination. Fabricated items can in principle be caught by checking each one against the source. Missing items cannot: verifying completeness means re-doing the whole query. That makes omission the more consequential failure for applications that act on the full result, such as notifying every affected customer or summing every qualifying transaction.

Model	Question	Answers	Records	Main-run outcome	Reproduced
Claude Haiku 5.5	`enum-par-013`	2	50	recall 0.50, 1 missing	2/3
Claude Haiku 5.5	`enum-par-021`	11	50	recall 0.73, 3 missing	3/3
Claude Haiku 5.5	`sel-029`	13	400	recall 0.85, 2 missing	0/3
Claude Haiku 5.5	`join-004`	55	100	recall 0.93, 4 missing	0/3
Claude Haiku 5.5	`nest-022`	13	200	recall 0.92, 1 missing	2/3
Claude Haiku 5.5	`proj-031`	37	300	recall 0.89, 4 missing	0/3
Claude Haiku 5.5	`agg-015`	12	400	answered 11 (gold 12)	0/3
Claude Haiku 5.5	`enum-ctx-029`	50	500	recall 0.96, 2 missing	0/3
DeepSeek V4 Pro	`enum-ctx-029`	50	500	recall 0.98, 1 missing	3/3
MiMo-V2.6 Flash	`enum-ctx-029`	50	500	recall 0.94, 3 missing, 3 wrong	2/3*
MiMo-V2.6 Pro	`enum-ctx-029`	50	500	recall 0.92, 4 missing	1/3
MiMo-V2.6 Flash	`enum-ctx-035`	100	1000	recall 0.52, 46 missing, 2 near-miss, 19 wrong, 371 made-up, 34 duplicate	2/3
DeepSeek V4 Flash	`enum-ctx-018`	5	50	recall 0.80, 1 near-miss	0/3

Table 6: RQ4. Selected failures from E1, each re-run three more times on the same model. *Reproduced* counts re-runs that were again not exactly correct; an asterisk marks a re-run with no parseable answer, which is not counted. Cases were selected during the run; the MiMo cases come from an earlier, partial OpenCode Go run of the same models (the E1 results above use OpenRouter), and the DeepSeek and MiMo re-runs used OpenRouter.

Model	Qs	Exhaustive − direct (pp)	Qs	Row-by-row − direct (pp)
DeepSeek V4 Pro	–	–	–	–
DeepSeek V4 Flash	93	−2.2 [-5.4, +0.0]	93	+0.0 [+0.0, +0.0]
Claude Haiku 5.5	93	+1.1 [-3.2, +6.5]	93	−2.2 [-6.5, +2.2]
MiMo-V2.6 Pro	93	+1.1 [-4.3, +6.5]	93	+4.3 [-1.1, +9.7]
MiMo-V2.6 Flash	93	+0.0 [-8.6, +9.7]	93	+6.5 [-2.2, +15.1]

Table 7: RQ5. Paired change in exact-set accuracy (percentage points) from the direct prompt, on set-valued questions of the 100-question subset, with 95% bootstrap intervals. DeepSeek V4 Pro's E2 runs did not complete within its provider's usage limits.

Risk concentrates in large answer sets. For three models, each doubling of the answer set cuts the odds of a complete answer by more than half. Applications that expect small results are largely safe with these models; applications that may return dozens of items are not, unless the query is executed by a database.

Prompting is not a fix; reasoning may be. Asking for exhaustiveness or a record-by-record check did not help. The models that stay complete appear to do so by reasoning at length, which costs tokens and latency: 77–97% of all output tokens were reasoning (Appendix Table 11). Whether forcing more reasoning closes the gap for weaker models is an open, testable question.

Feasibility of a full benchmark. The phenomenon is real, measurable with exact ground truth, and separates models: three of the five trail the best model by 7–17 percentage points, with intervals well away from zero. At the same time, the strongest models are close to ceiling on this version, whose largest answer set has 100 items. A full benchmark would need harder tiers (larger answer sets and tables, a reasoning-disabled condition, more distractors) to keep separating frontier models.

7 Limitations

- **Synthetic data and one seed.** Records are generated from templates; results may differ on real, messier data. All questions come from one generation seed.
- **Single samples in E1.** Each question was answered once in the main run; E3 shows that 0–13% of verdicts change between samples, so individual results should not be over-interpreted.

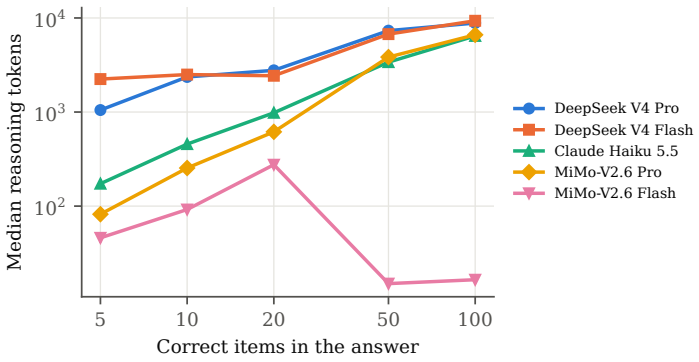


Figure 3: RQ5. Median reasoning tokens on in-context enumeration against the number of correct items (E1).

- **Retrieval slice.** With a unique category keyword, BM25 returns matching records first, so the retrieval slice measured the retriever rather than the model.
- **Access differences.** Models were reached through different providers, and Claude Haiku 5.5 through a CLI without temperature control; each model's reasoning was left at its default.
- **Incomplete runs.** DeepSeek V4 Pro's prompt-variant runs (E2) did not finish under its provider's usage limits, and a Claude Haiku 4.5 run was stopped after 90 questions; both are excluded.
- **Strict identifiers.** Items must be named exactly (up to normalisation). One MiMo-V2.6 Flash answer that dropped product model codes was scored as wrong, because the shortened names were ambiguous.

8 Conclusion

Models that look nearly perfect by recall return incomplete answers to between 2% and 19% of "list all" questions, and the rate rises with the number of correct items. The benchmark isolates this failure with exact ground truth, separates models with statistically clear margins, and shows that simple prompting does not remove it. Natural next steps are harder tiers to restore headroom at the frontier, a controlled manipulation of reasoning effort, real-world datasets, and a direct comparison with HoloBench [10].

Reproducibility. Code, the benchmark (500 questions and 575 tables), every raw response with token usage and timestamps, and per-run manifests recording the code commit (the first run used `09d66f2`), the benchmark hash (`b2bc2242be80...`) and software versions are available at `https://github.com/daryl-888/llm-relational-completeness`. Every table and figure in this report is generated from those files by `report/make_assets.py`.

References

[1] LongProc: Benchmarking long-context language models on long procedural generation. *arXiv preprint arXiv:2501.05414*, 2025.

[2] Iyad Ait Hou, Saad Mankarious, Aya Zirikly, and Rebecca Hwa. List counting failures are not one phenomenon. *arXiv preprint arXiv:2609.22230*, 2026.

[3] Samuel Joseph Amouyal et al. QAMPARI: A benchmark for open-domain questions with many answers. In *Proceedings of the Third Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, 2023.

[4] Amanda Bertsch, Adithya Pratapa, Teruko Mitamura, Graham Neubig, and Matthew R. Gormley. Oolong: Evaluating long context reasoning and aggregation capabilities. *arXiv preprint arXiv:2511.02817*, 2025.

[5] Asim Biswal et al. Text2SQL is not enough: Unifying AI and databases with TAG. *Conference on Innovative Data Systems Research (CIDR)*, 2025.

[6] Cheng-Ping Hsieh et al. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[7] Yuri Kuratov et al. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[8] Jinhyuk Lee, Anthony Chen, Zhuyun Dai, Dheeru Dua, et al. Can long-context language models subsume retrieval, RAG, SQL, and more? *arXiv preprint arXiv:2406.13121*, 2024.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.

[10] Seiji Maekawa, Hayate Iso, and Nikita Bhutani. Holistic reasoning with long-context LMs: A benchmark for database operations on massive textual data. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.11996.

[11] Chaitanya Malaviya, Peter Shaw, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. QUEST: A retrieval dataset of entity-seeking queries with implicit set operations. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[12] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.

[13] Liana Patel et al. Semantic operators: A declarative model for rich, AI-based data processing. *Proceedings of the VLDB Endowment*, 18, 2025.

[14] Anka Reuel, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, and Mykel J. Kochenderfer. BetterBench: Assessing AI benchmarks, uncovering issues, and establishing best practices. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[15] Mohammed Saeed, Nicola De Cao, and Paolo Papotti. Querying large language models with SQL. In *International Conference on Extending Database Technology (EDBT)*, 2024.

[16] Damien Sileo. Attention overflow: Language model input blur during long-context missing items recommendation. *arXiv preprint arXiv:2407.13481*, 2024.

[17] Zhanqi Zhang, Hua-Dong Xiong, Robert C. Wilson, Mikio Aoi, Marcelo G. Mattar, and Li Ji-An. The position curse: LLMs struggle to locate the last few items in a list. *arXiv preprint arXiv:2605.07127*, 2026.

[18] Victor Zhong et al. RoMQA: A benchmark for robust, multi-evidence, multi-answer question answering. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

A Additional results

Model	Missing	Near-miss	Wrong	Made-up	Duplicate
DeepSeek V4 Pro	16	0	1	0	0
DeepSeek V4 Flash	43	1	3	0	0
Claude Haiku 5.5	75	0	6	0	0
MiMo-V2.6 Pro	104	0	15	0	0
MiMo-V2.6 Flash	492	7	311	1,032	316

Table 8: Error counts by type on in-context set-valued questions (E1).

Model	Memory: exact (%)	Memory: recall	Retrieval: recall	Retrieval: recall over retrieved
DeepSeek V4 Pro	100	1.000	0.869	1.000
DeepSeek V4 Flash	100	1.000	0.869	1.000
Claude Haiku 5.5	87	0.952	0.869	1.000
MiMo-V2.6 Pro	83	0.955	0.869	1.000
MiMo-V2.6 Flash	83	0.958	0.869	1.000

Table 9: Memory-only and retrieval settings (E1). "Recall over retrieved" counts only the correct items that the retriever returned.

Model	Qs	Exact (%)	Under-counts	Over-counts
DeepSeek V4 Pro	35	100	0	0
DeepSeek V4 Flash	35	100	0	0
Claude Haiku 5.5	35	94	2	0
MiMo-V2.6 Pro	35	91	3	0
MiMo-V2.6 Flash	35	83	4	2

Table 10: Scalar aggregation questions (COUNT, SUM, MIN, MAX, AVG; E1).

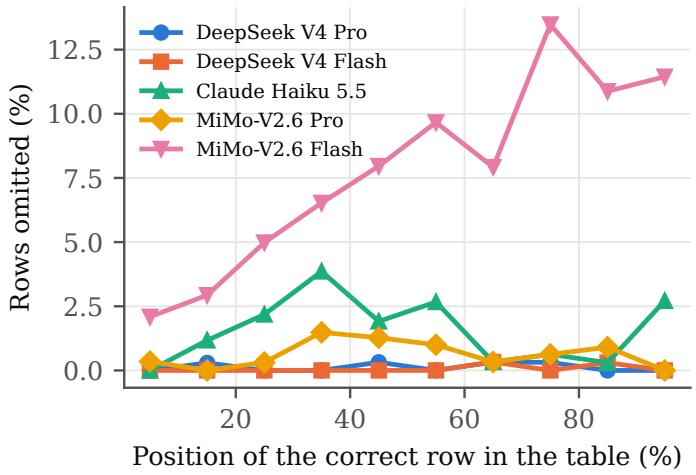


Figure 4: Share of correct rows omitted, by the row's position in the table (in-context enumeration, E1). MiMo-V2.6 Flash omits rows near the end of the table more often; the other models omit too few rows for a clear pattern, and no model shows a U-shaped "lost in the middle" profile [9].

Model	Access	Calls	Input (M)	Output (M)	Reasoning (%)	Cost (USD)	Latency (s)
DeepSeek V4 Pro	OpenCode Go	1,027	3.83	2.21	97	6.87	23.4
DeepSeek V4 Flash	OpenCode Go	1,307	5.08	3.02	93	2.52	12.5
Claude Haiku 5.5	Claude Code CLI	893	8.04	1.88	87	2.49	7.7
MiMo-V2.6 Pro	OpenRouter	800	5.77	2.00	89	3.29	16.6
MiMo-V2.6 Flash	OpenRouter	800	5.78	1.72	77	1.25	13.1

Table 11: Usage across all experiments (4,827 calls, including retried calls). Input and output are millions of tokens; *Reasoning* is the share of output tokens spent on reasoning; cost is at provider list prices (Claude Haiku 5.5 and the OpenCode Go models were accessed through subscriptions); latency is the median per call.